

ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 27

Брокеры сообщений: RabbitMQ и Kafka

Рассмотрим паттерн Message Queue, принципы работы брокеров сообщений RabbitMQ и Kafka, архитектуру Producer-Consumer, организацию воркер-процессов, а также практики повышения надёжности: retry logic с экспоненциальной задержкой и Dead Letter Queue. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Паттерн Message Queue

При разработке распределённых систем возникает необходимость передавать данные между независимыми сервисами. Прямые синхронные вызовы (HTTP REST) создают жёсткую связанность: если сервис-получатель недоступен, вызывающий сервис получит ошибку и данные будут потеряны.

Паттерн Message Queue (очередь сообщений) решает эту проблему: вместо прямого вызова сервис-отправитель помещает сообщение в очередь, а сервис-получатель забирает его тогда, когда готов. Между ними стоит **брокер сообщений** — посредник, хранящий и доставляющий сообщения.

Ключевые преимущества:

- **Асинхронность** — отправитель не ждёт ответа получателя.
- **Буферизация нагрузки** — очередь поглощает пики трафика; воркеры обрабатывают сообщения в своём темпе.
- **Надёжность** — если получатель упал, сообщения сохраняются в очереди и будут доставлены после восстановления.
- **Слабая связанность** — сервисы не знают друг о друге; они знают только о брокере.

Типичные сценарии применения:

- отправка email/SMS-уведомлений после регистрации пользователя;
- обработка платёжных транзакций;
- генерация отчётов и экспорт данных;
- обработка загруженных файлов (изображений, видео);
- распределение задач между несколькими воркерами.

RabbitMQ

RabbitMQ — один из наиболее распространённых брокеров сообщений, реализующий протокол **AMQP** (Advanced Message Queuing Protocol). Разработан в 2007 году и широко используется в enterprise-системах.

Ключевые концепции RabbitMQ

Producer (Производитель) — приложение, которое публикует сообщения. Producer никогда не отправляет сообщение напрямую в очередь — только в Exchange.

Exchange (Обменник) — принимает сообщения от Producer и маршрутизирует их в одну или несколько очередей по правилам. Типы Exchange:

- **direct** — доставляет в очередь, у которой routing key совпадает с ключом сообщения;
- **fanout** — рассылает сообщение во все привязанные очереди (broadcast);
- **topic** — маршрутизация по шаблонам (`*.error` , `payment.#`);
- **headers** — маршрутизация по заголовкам сообщения.

Queue (Очередь) — буфер хранения сообщений. Сообщения хранятся до тех пор, пока Consumer не заберёт и не подтвердит их получение.

Consumer (Потребитель) — приложение, которое получает и обрабатывает сообщения из очереди.

Binding — связь между Exchange и Queue с указанием routing key.

Архитектура RabbitMQ

```
Producer → Exchange → [Binding + Routing Key] → Queue → Consumer
```

Установка RabbitMQ через Docker

```
docker run -d \  
  --name rabbitmq \  
  -p 5672:5672 \
```

```
-p 15672:15672 \  
rabbitmq:3-management
```

- Порт 5672 — AMQP (подключение приложений).
- Порт 15672 — веб-интерфейс управления (логин/пароль по умолчанию: `guest / guest`).

Работа с RabbitMQ в Node.js

```
npm install amqplib
```

Producer — отправка сообщений:

```
// producer.js  
import amqplib from 'amqplib';  
  
async function sendMessage(queueName, data) {  
  const connection = await amqplib.connect('amqp://localhost');  
  const channel = await connection.createChannel();  
  
  // Убеждаемся, что очередь существует (idempotent)  
  await channel.assertQueue(queueName, { durable: true });  
  
  const message = JSON.stringify(data);  
  
  channel.sendToQueue(  
    queueName,  
    Buffer.from(message),  
    { persistent: true } // Сообщение переживёт перезапуск RabbitMQ  
  );  
  
  console.log(`[Producer] Отправлено: ${message}`);  
  
  setTimeout(() => connection.close(), 500);  
}  
  
await sendMessage('email_queue', {  
  to: 'user@example.com',  
  subject: 'Добро пожаловать!',  
  body: 'Спасибо за регистрацию.',  
});
```

Consumer — получение и обработка сообщений:

```
// consumer.js  
import amqplib from 'amqplib';
```

```

async function startConsumer(queueName) {
  const connection = await amqpLib.connect('amqp://localhost');
  const channel = await connection.createChannel();

  await channel.assertQueue(queueName, { durable: true });

  // Обрабатываем по одному сообщению за раз
  channel.prefetch(1);

  console.log(`[Consumer] Ожидание сообщений в "${queueName}"...`);

  channel.consume(queueName, async (msg) => {
    if (!msg) return;

    const data = JSON.parse(msg.content.toString());
    console.log(`[Consumer] Получено:`, data);

    try {
      // Имитация обработки (отправка email)
      await processEmail(data);

      // Подтверждаем успешную обработку
      channel.ack(msg);
    } catch (err) {
      console.error(`[Consumer] Ошибка обработки:`, err.message);
      // Возвращаем сообщение в очередь (requeue: true)
      channel.nack(msg, false, true);
    }
  });
}

async function processEmail(data) {
  // В реальном проекте — вызов email-сервиса
  console.log(`Отправка email на ${data.to}: ${data.subject}`);
  await new Promise(resolve => setTimeout(resolve, 1000));
}

await startConsumer('email_queue');

```

Apache Kafka

Apache Kafka — распределённая платформа потоковой обработки данных, разработанная в LinkedIn в 2010 году. В отличие от RabbitMQ, Kafka оптимизирована для **высокой пропускной способности** и **долгосрочного хранения** потоков событий.

Ключевые концепции Kafka

Topic (Тема) — категория, в которую Producer публикует сообщения. Consumer читает из топика. Аналог очереди в RabbitMQ, но с важным отличием: сообщения не

удаляются после прочтения.

Partition (Раздел) — каждый топик разбит на разделы. Разделы позволяют параллельно читать данные разными Consumer'ами и распределять нагрузку по нескольким брокерам.

Offset (Смещение) — порядковый номер сообщения внутри раздела. Consumer запоминает свой offset и может перечитать историю сообщений.

Consumer Group (Группа потребителей) — несколько Consumer'ов объединяются в группу. Kafka автоматически распределяет разделы топика между членами группы, обеспечивая параллельную обработку.

Retention — срок хранения сообщений (по умолчанию 7 дней). По истечении срока сообщения удаляются независимо от того, были ли они прочитаны.

Установка Kafka через Docker Compose

```
# docker-compose.yml
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

```
docker compose up -d
```

Работа с Kafka в Node.js

```
npm install kafkajs
```

Producer:

```
// kafka-producer.js
import { Kafka } from 'kafkajs';

const kafka = new Kafka({
  clientId: 'my-app',
  brokers: ['localhost:9092'],
});

const producer = kafka.producer();

async function sendEvent(topic, event) {
  await producer.connect();

  await producer.send({
    topic,
    messages: [
      {
        key: event.userId, // Ключ гарантирует порядок
        value: JSON.stringify(event),
      },
    ],
  });

  console.log(`[Kafka Producer] Событие отправлено в топик "${topic}"`);
  await producer.disconnect();
}

await sendEvent('user-events', {
  userId: 'user-42',
  action: 'purchase',
  productId: 'prod-100',
  amount: 1500,
  timestamp: new Date().toISOString(),
});
```

Consumer:

```
// kafka-consumer.js
import { Kafka } from 'kafkajs';

const kafka = new Kafka({
  clientId: 'my-app',
  brokers: ['localhost:9092'],
});

const consumer = kafka.consumer({ groupId: 'analytics-service' });
```

```

async function startConsumer() {
    await consumer.connect();
    await consumer.subscribe({ topic: 'user-events', fromBeginning: false
});

    await consumer.run({
        eachMessage: async ({ topic, partition, message }) => {
            const event = JSON.parse(message.value.toString());
            console.log(`[Kafka Consumer] Получено из ${topic}
[${partition}]:`, event);

            // Обработка события
            await processUserEvent(event);
        },
    });
}

async function processUserEvent(event) {
    console.log(`Аналитика: пользователь ${event.userId} выполнил
${event.action}`);
}

await startConsumer();

```

Сравнение RabbitMQ и Kafka

Характеристика	RabbitMQ	Apache Kafka
Парадигма	Message Broker (push)	Event Streaming Platform (pull)
Хранение сообщений	Удаляются после подтверждения	Хранятся заданное время (retention)
Порядок сообщений	В рамках одной очереди	В рамках одного раздела (partition)
Пропускная способность	Высокая (~50K сообщений/с)	Очень высокая (~1M+ сообщений/с)
Маршрутизация	Гибкая (Exchange, Binding, Routing Key)	Только по топикам и разделам
Повторное чтение	Не поддерживается	Поддерживается (через offset)
Сложность настройки	Низкая	Средняя и выше
Типичное применение	Задачи, уведомления, RPC	Логи, события, потоковая аналитика



Когда выбрать RabbitMQ: задачи на обработку (отправка email, генерация PDF), когда важна гибкая маршрутизация и простота настройки.

Когда выбрать Kafka: когда важна высокая пропускная способность, история событий (event sourcing), потоковая обработка данных.

Retry Logic и экспоненциальная задержка

В реальных системах обработка сообщений может завершиться с ошибкой (внешний сервис недоступен, временный сбой). Немедленная повторная попытка часто не помогает — сервис ещё не успел восстановиться. Правильный подход — **экспоненциальная задержка** (exponential backoff): каждая следующая попытка ждёт вдвое дольше предыдущей, плюс случайный «джиттер» для предотвращения одновременной нагрузки.

```
// retry.js
async function processWithRetry(message, processor, options = {}) {
  const {
    maxRetries = 5,
    baseDelayMs = 1000,
    maxDelayMs = 30000,
  } = options;

  let attempt = 0;

  while (attempt <= maxRetries) {
    try {
      await processor(message);
      console.log(`[Retry] Успешно обработано за ${attempt + 1}
попытку(и)`);
      return;
    } catch (err) {
      attempt++;

      if (attempt > maxRetries) {
        console.error(`[Retry] Искерпаны все ${maxRetries} попытки.
Сообщение отправляется в DLQ.`);
        throw err; // Пробрасываем ошибку – Consumer направит в DLQ
      }

      // Экспоненциальная задержка с джиттером
      const exponentialDelay = Math.min(baseDelayMs * 2 ** (attempt -
1), maxDelayMs);
      const jitter = Math.random() * 1000;
      const delay = exponentialDelay + jitter;
```

```

        console.warn(`[Retry] Попытка ${attempt}/${maxRetries}
провалилась: ${err.message}. Повтор через ${Math.round(delay)}ms`);
        await new Promise(resolve => setTimeout(resolve, delay));
    }
}

// Использование
await processWithRetry(
    { to: 'user@example.com', subject: 'Test' },
    async (msg) => {
        // Имитация нестабильного внешнего сервиса
        if (Math.random() < 0.7) throw new Error('Email service
unavailable');
        console.log('Email отправлен на ${msg.to}');
    }
);

```

Dead Letter Queue (DLQ)

Dead Letter Queue (очередь мёртвых писем) — это специальная очередь, в которую попадают сообщения, которые не удалось обработать даже после всех попыток повтора. DLQ позволяет:

- не терять сообщения при неустранимых ошибках;
- анализировать проблемные сообщения вручную;
- при необходимости повторно запускать их в обработку.

Настройка DLQ в RabbitMQ

```

// setup-queues.js
import amqplib from 'amqplib';

async function setupQueues() {
    const connection = await amqplib.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // 1. Создаём Dead Letter Exchange
    await channel.assertExchange('dlx_exchange', 'direct', { durable: true
});

    // 2. Создаём Dead Letter Queue
    await channel.assertQueue('dead_letter_queue', { durable: true });

    // 3. Привязываем DLQ к DLX
    await channel.bindQueue('dead_letter_queue', 'dlx_exchange', 'dead');
}

```

```
// 4. Создаём основную очередь с указанием DLX
await channel.assertQueue('main_queue', {
  durable: true,
  arguments: {
    'x-dead-letter-exchange': 'dlx_exchange',
    'x-dead-letter-routing-key': 'dead',
    'x-message-ttl': 60000,      // Сообщение живёт 60 секунд
    'x-max-retries': 3,         // (информационно, логика retry – в
    // коде Consumer)
  },
});

console.log('Очереди настроены: main_queue → [DLX] →
dead_letter_queue');
await connection.close();
}

await setupQueues();
```

Consumer с поддержкой DLQ

```
// consumer-with-dlq.js
import amqplib from 'amqplib';

const MAX_RETRIES = 3;

async function startConsumerWithDLQ() {
  const connection = await amqplib.connect('amqp://localhost');
  const channel = await connection.createChannel();

  channel.prefetch(1);

  channel.consume('main_queue', async (msg) => {
    if (!msg) return;

    const data = JSON.parse(msg.content.toString());
    const retryCount = (msg.properties.headers?.['x-retry-count'] || 0);

    console.log(`[Consumer] Попытка ${retryCount + 1}/${MAX_RETRIES}:`,
data);

    try {
      await processMessage(data);
      channel.ack(msg);
    } catch (err) {
      console.error(`[Consumer] Ошибка: ${err.message}`);

      if (retryCount < MAX_RETRIES) {
        // Повторная публикация с увеличенным счётчиком

```

```

        channel.nack(msg, false, false); // Отклоняем без возврата
        в очередь

        const delay = 1000 * 2 ** retryCount;
        await new Promise(resolve => setTimeout(resolve, delay));

        channel.sendToQueue('main_queue', msg.content, {
            persistent: true,
            headers: { 'x-retry-count': retryCount + 1 },
        });
    } else {
        // Исчерпали попытки — сообщение уйдёт в DLQ через nack
        console.error(`[Consumer] Сообщение отправлено в DLQ после
        ${MAX_RETRIES} попыток`);
        channel.nack(msg, false, false);
    }
}
});
}

async function processMessage(data) {
    if (Math.random() < 0.8) throw new Error('Processing failed');
    console.log('[Consumer] Сообщение успешно обработано:', data);
}

await startConsumerWithDLQ();

```

Worker-процессы

Паттерн **Worker** (рабочий процесс) предполагает запуск нескольких экземпляров Consumer для параллельной обработки сообщений из одной очереди. RabbitMQ и Kafka автоматически распределяют сообщения между воркерами — каждое сообщение обрабатывается ровно одним воркером.

```

// Запуск нескольких воркеров
// worker.js — один файл, запускается несколько раз
const WORKER_ID = process.env.WORKER_ID || '1';

async function startWorker() {
    const connection = await amqplib.connect('amqp://localhost');
    const channel = await connection.createChannel();

    channel.prefetch(1); // Каждый воркер берёт по одному сообщению

    channel.consume('task_queue', async (msg) => {
        const task = JSON.parse(msg.content.toString());
        console.log(`[Worker ${WORKER_ID}] Обработка задачи: ${task.id}`);
    });
}

```

```
    await performTask(task);
    channel.ack(msg);

    console.log(`[Worker ${WORKER_ID}] Задача ${task.id} выполнена`);
  });

  console.log(`[Worker ${WORKER_ID}] Запущен, ожидание задач...`);
}

await startWorker();
```

```
# Запуск трёх воркеров в параллельных терминалах
WORKER_ID=1 node worker.js
WORKER_ID=2 node worker.js
WORKER_ID=3 node worker.js
```

Практическое задание

Необходимо реализовать систему асинхронной обработки задач с использованием RabbitMQ.

В рамках выполнения задания требуется:

- запустить RabbitMQ через Docker;
- реализовать **Producer** — Express API с маршрутом `POST /tasks`, который принимает задачу (например, `{ type: "email", payload: {...} }`) и помещает её в очередь;
- реализовать **Consumer** (воркер), который читает задачи из очереди и «обрабатывает» их (вывод в консоль с задержкой, имитирующей работу);
- добавить **Retry Logic** с экспоненциальной задержкой (минимум 3 попытки);
- настроить **Dead Letter Queue** для сообщений, которые не удалось обработать;
- запустить минимум **два воркера** одновременно и убедиться, что задачи распределяются между ними.

Формат отчета

В качестве ответа на данный блок практик студентом подготавливается тематический проект. Критерии в [Практике 28](#)

Литература

1. [Документация RabbitMQ](#)
2. [Библиотека amqp-lib для Node.js](#)
3. [Документация Apache Kafka](#)

4. [Библиотека KafkaJS](#)